Open in Colab

Module AA: Association Analysis

An example from this link.

https://www.kaggle.com/code/sangwookchn/association-rule-learning-with-scikit-learn/notebook

```
# Please install the package. There is another package class mlxend.
!pip install apyori
```

```
Collecting apyori
  Downloading apyori-1.1.2.tar.gz (8.6 kB)
  Preparing metadata (setup.py) ... done
Building wheels for collected packages: apyori
  Building wheel for apyori (setup.py) ... done
  Created wheel for apyori: filename=apyori-1.1.2-py3-none-any.whl size=5954 sha256=75aacf193d122a77a81ce4598c50589d19d0801ebd222c60e35d542c4d9c923
  Stored in directory: /root/.cache/pip/wheels/7f/49/e3/42c73b19a264de37129fadaa0c52f26cf50e87de08fb9804af
Successfully built apyori
Installing collected packages: apyori
Successfully installed apyori-1.1.2
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from apyori import apriori
```

```
dataset = pd.read_csv('http://pluto.hood.edu/~dong/datasets/Market_Basket_Optimisation.csv', header = None) #To make sure the first row is not tho
print(dataset.shape)
dataset.head(10)
```

```
(7501, 20)
```

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
0	shrimp	almonds	avocado	vegetables mix	green grapes	whole weat flour	yams	cottage cheese	energy drink	tomato juice	low fat yogurt	green tea	honey	salad	mineral water	salmon	antioxydant juice	frozen smoothie
1	burgers	meatballs	eggs	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	chutney	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	turkey	avocado	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	mineral water	milk	energy bar	whole wheat rice	green tea	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
5	low fat yogurt	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
6	whole wheat pasta	french fries	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

Next steps: Generate code with `dataset` New interactive sheet

```
#Transforming the list into a list of lists, so that each transaction can be indexed easier
transactions = []
for i in range(0, dataset.shape[0]):
    transactions.append([str(dataset.values[i, j]) for j in range(0, 20)])

print(transactions[0:2])
```

```
[['shrimp', 'almonds', 'avocado', 'vegetables mix', 'green grapes', 'whole weat flour', 'yams', 'cottage cheese', 'energy drink', 'tomato juice', '
```

```
# Support: number of transactions containing set of times / total number of transactions
# .      --> products that are bought at least 3 times a day --> 21 / 7501 = 0.0027
# Confidence: Should not be too high, as then this wil lead to obvious rules
#Try many combinations of values to experiment with the model.

rules = apriori(transactions, min_support = 0.003, min_confidence = 0.2, min_lift = 3, min_length = 2)

#viewing the rules
results = list(rules)
```

```
#Transferring the list to a table

results = pd.DataFrame(results)
```

```
results.head(10)
```

	items	support	ordered_statistics
0	(chicken, light cream)	0.004533	[((light cream), (chicken), 0.2905982905982905...
1	(escalope, mushroom cream sauce)	0.005733	[((mushroom cream sauce), (escalope), 0.300699...
2	(escalope, pasta)	0.005866	[((pasta), (escalope), 0.3728813559322034, 4.7...
3	(honey, fromage blanc)	0.003333	[((fromage blanc), (honey), 0.2450980392156863...
4	(ground beef, herb & pepper)	0.015998	[((herb & pepper), (ground beef), 0.3234501347...
5	(ground beef, tomato sauce)	0.005333	[((tomato sauce), (ground beef), 0.37735849056...
6	(olive oil, light cream)	0.003200	[((light cream), (olive oil), 0.20512820512820...
7	(olive oil, whole wheat pasta)	0.007999	[((whole wheat pasta), (olive oil), 0.27149321...
8	(shrimp, pasta)	0.005066	[((pasta), (shrimp), 0.3220338983050847, 4.506...
9	(milk, spaghetti, avocado)	0.003333	[((spaghetti, avocado), (milk), 0.416666666666...

Next steps: Generate code with `results` New interactive sheet

"The first item in the list is a list itself containing three items. The first item of the list shows the grocery items in the rule.

For instance from the first item, we can see that light cream and chicken are commonly bought together. This makes sense since people who purchase light cream are careful about what they eat hence they are more likely to buy chicken i.e. white meat instead of red meat i.e. beef. Or this could mean that light cream is commonly used in recipes for chicken.

The support value for the first rule is 0.0045. This number is calculated by dividing the number of transactions containing light cream divided by total number of transactions. The confidence level for the rule is 0.2905 which shows that out of all the transactions that contain light cream, 29.05% of the transactions also contain chicken. Finally, the lift of 4.84 tells us that chicken is 4.84 times more likely to be bought by the customers who buy light cream compared to the default likelihood of the sale of chicken."

From https://stackabuse.com/association-rule-mining-via-apriori-algorithm-in-python/

In-Class: Association Analysis

Customer Computer Configuration

```
pc_purchase = pd.read_csv('http://pluto.hood.edu/~dong/datasets/PC-Purchase-Data.csv') #To make sure the first row is not thought of as the headin
print(pc_purchase.shape)
pc_purchase.head()
```

```
(67, 12)
```

	Intel Core i3	Intel Core i5	Intel Core i7	10 inch screen	12 inch screen	15 inch screen	2 GB	4 GB	8 GB	320 GB	500 GB	750 GB
0	0	1	0	0	1	0	0	1	0	0	1	0
1	0	1	0	0	0	1	0	0	1	0	0	1
2	0	1	0	0	1	0	0	1	0	1	0	0
3	1	0	0	0	1	0	0	0	1	0	1	0
4	0	0	1	0	0	1	0	0	1	0	0	1

Next steps: Generate code with `pc_purchase` New interactive sheet

The data represent the configurations for a small number of orders of laptops placed over the web. The main options from which customers can choose are the type of processors, screen size, memory, and hard drive. A '1' signifies that a customer selected a particular option. If the manufacturer can better understand what types of components are often ordered together, it can speed up final assembly by having partially completed laptops with the mYourorelar combinations of orderingnents configured before order.

```
Start coding or generate with AI.
```

You task is to find the hidden pattern with respect to popular configuraions. Use support, confidence, and lift correctly to explain your findings.

Hint: make sure the data are in the right format.

```
transactions = []
for i in range(0, pc_purchase.shape[0]):
    transaction = []
```

```
    for j in range(0, pc_purchase.shape[1]):
        if pc_purchase.values[i, j] == 1:
            transaction.append(pc_purchase.columns[j])
    transactions.append(transaction)

print(transactions[0:5])
```

```
[['Intel Core i5', '12 inch screen', '4 GB', '500 GB'], ['Intel Core i5', '15 inch screen', '8 GB', '750 GB'], ['Intel Core i5', '12 inch screen',
```

```
from apyori import apriori

rules = apriori(transactions, min_support = 0.1, min_confidence = 0.5, min_lift = 2)

results = list(rules)

print(len(results))
results_df = pd.DataFrame(results)
print(results_df.shape)
display(results_df.head())
```

```
2
(2, 3)
```

	items	support	ordered_statistics
0	(8 GB, 750 GB)	0.104478	[((8 GB), (750 GB), 0.5384615384615384, 2.1221...
1	(750 GB, Intel Core i7)	0.119403	[((Intel Core i7), (750 GB), 0.666666666666666...

```
rules_experiment = apriori(transactions, min_support = 0.05, min_confidence = 0.2, min_lift = 1)
results_experiment = list(rules_experiment)


print(f"num of rules with min_support=0.05, min_confidence=0.2, min_lift=1: {len(results_experiment)}")
results_experiment_df = pd.DataFrame(results_experiment)
display(results_experiment_df.head())

rules_experiment_2 = apriori(transactions, min_support = 0.15, min_confidence = 0.7, min_lift = 4)
results_experiment_2 = list(rules_experiment_2)
print(f"num of rules with min_support=0.15, min_confidence=0.7, min_lift=4: {len(results_experiment_2)}")
results_experiment_2_df = pd.DataFrame(results_experiment_2)
display(results_experiment_2_df.head())
```

```
Number of rules with min_support=0.05, min_confidence=0.2, min_lift=1: 77
```

	items	support	ordered_statistics
0	(10 inch screen)	0.238806	[((), (10 inch screen), 0.23880597014925373, 1...
1	(12 inch screen)	0.477612	[((), (12 inch screen), 0.47761194029850745, 1...
2	(15 inch screen)	0.283582	[((), (15 inch screen), 0.2835820895522388, 1.0)]
3	(2 GB)	0.253731	[((), (2 GB), 0.2537313432835821, 1.0)]
4	(320 GB)	0.283582	[((), (320 GB), 0.2835820895522388, 1.0)]

```
Number of rules with min_support=0.15, min_confidence=0.7, min_lift=4: 0
```

Interpretation of PC Purchase Association Rules

Based on the Apriori analysis with the following perameters min_support = 0.1, min_confidence = 0.5, and min_lift = 2, I found the following results:

1. **Rule:** 8 GB => 750 GB
    - Support: 0.1045 (appears in about 10.5% of transactions)
    - Confidence:0.5385 (when 8 GB RAM is purchased, 750 GB Hard Drive is also purchased about 53.9% of the time)
    - Lift: 2.1221 (customers who buy 8 GB RAM are about 2.12 times more likely to buy a 750 GB Hard Drive compared to the average customer)
    - personal interpretation of those results: There is a strong positive association between purchasing 8 GB of RAM and a 750 GB hard drive. This suggests that customers opting for higher memory tend to also choose larger storage.
2. Rule:Intel Core i7 => 750 GB
    - Support: 0.1194 (appears in about 11.9% of transactions)
    - Confidence: 0.6667 (when an Intel Core i7 processor is purchased a 750 GB Hard Drive is also purchased about 66.7% of the time.)
    - Lift: 2.6306 (customers who buy an Intel Core i7 processor are about 2.63 times more likely to buy a 750 GB Hard Drive compared, to the average customer)

- personal Interpretation of results: There is an even stronger positive association between purchasing an Intel Core i7 processor and a 750 GB hard drive. This indicates that customers buying the highestend processor frequently pair it with the largest available hard drive option.

These findings suggest that for the PC purchase data, customers buying higher-performance components 8 GB RAM, Intel Core i7 are strongly associated with also buying the largest hard drive 750 Gb.

Experimenting with different min_support, min_confidence, and min_lift values can reveal different levels of association. Lowering the thresholds (as seen in results_experiment_d) results in many more rules, including single item frequencies, while increasing them significantly (as seen in results_experiment_2_df) can result in no rules if the dataset is small or the associations are not yet strong enough to meet the criteria defined. The choice of thresholds depends on the specific business objective and the desired level of detail in the analysis we want to conduct

```
Start coding or generate with AI.
```